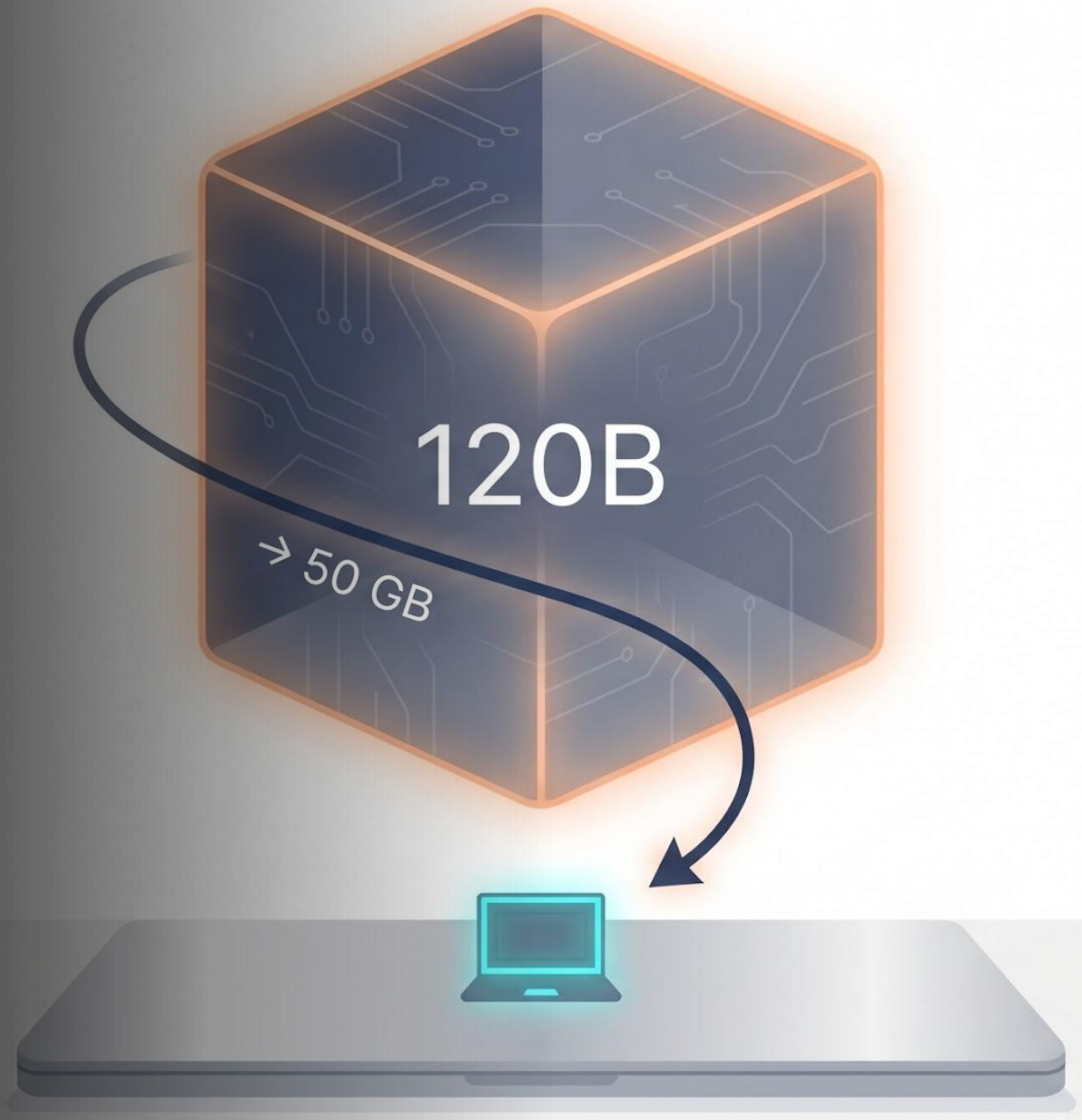
From 240 GB to 50 GB

Running Nemotron-3 Super 120B on a 64 GB MacBook with TurboQuant-MLX

Manjunath Janardhan · AI/ML Computational Science Senior Manager - Accenture



Who Am I

Manjunath Janardhan — AI/ML Computational Science Senior Manager – Accenture

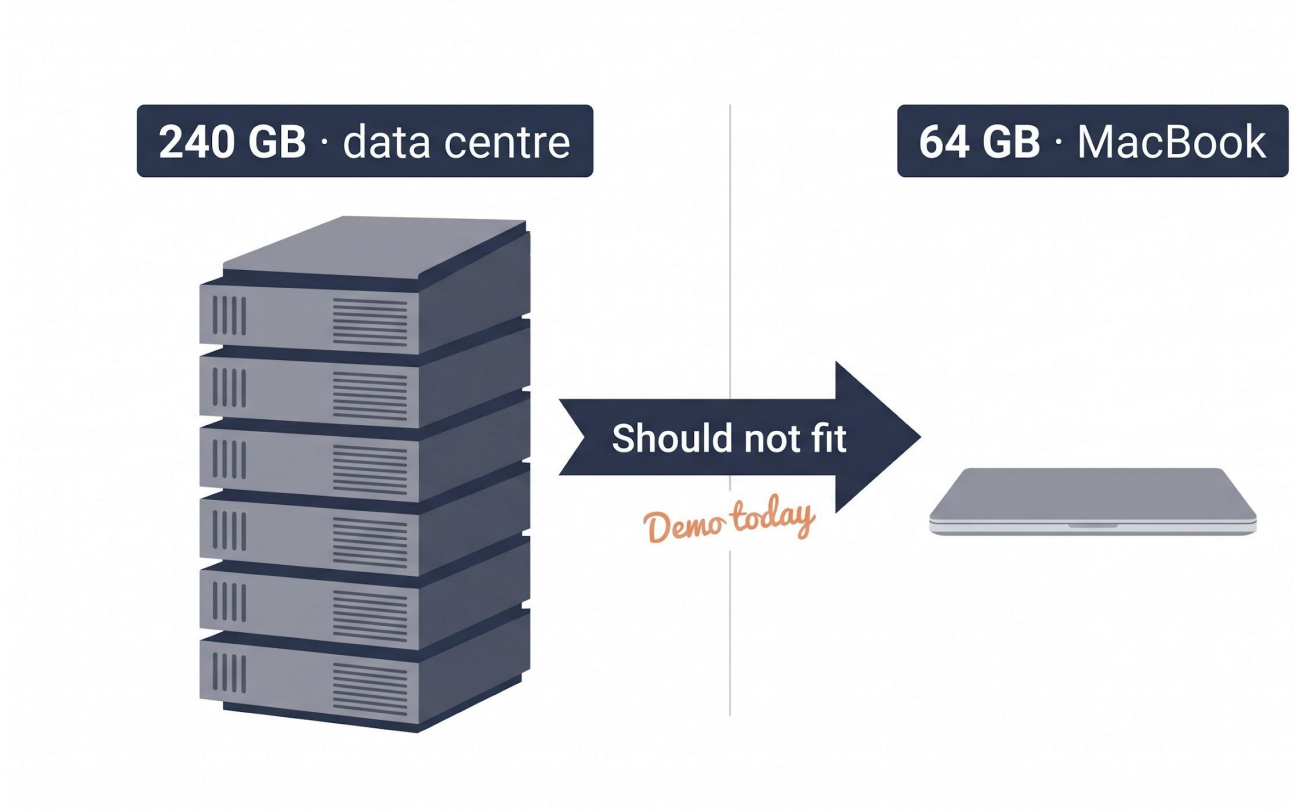
21+ years of Industry Experience

Agentic AI, Microservices, Multi-Cloud, C/C++/Java/Python

Research focus: making very large models run on small hardware

The Claim

- NVIDIA Nemotron-3 Super 120B → ~240 GB (native BF16)
- My MacBook → 64 GB of unified memory
- That's a 4× gap. In theory, it should not fit.
- Today we'll run it — live — in ~50 GB, at ~19 tokens/second.



Why This Matters

Big models live in data centers — and they should.

But developers iterate in small loops: try → break → fix → try again.

A laptop-scale dev loop means experimenting without burning cluster time.

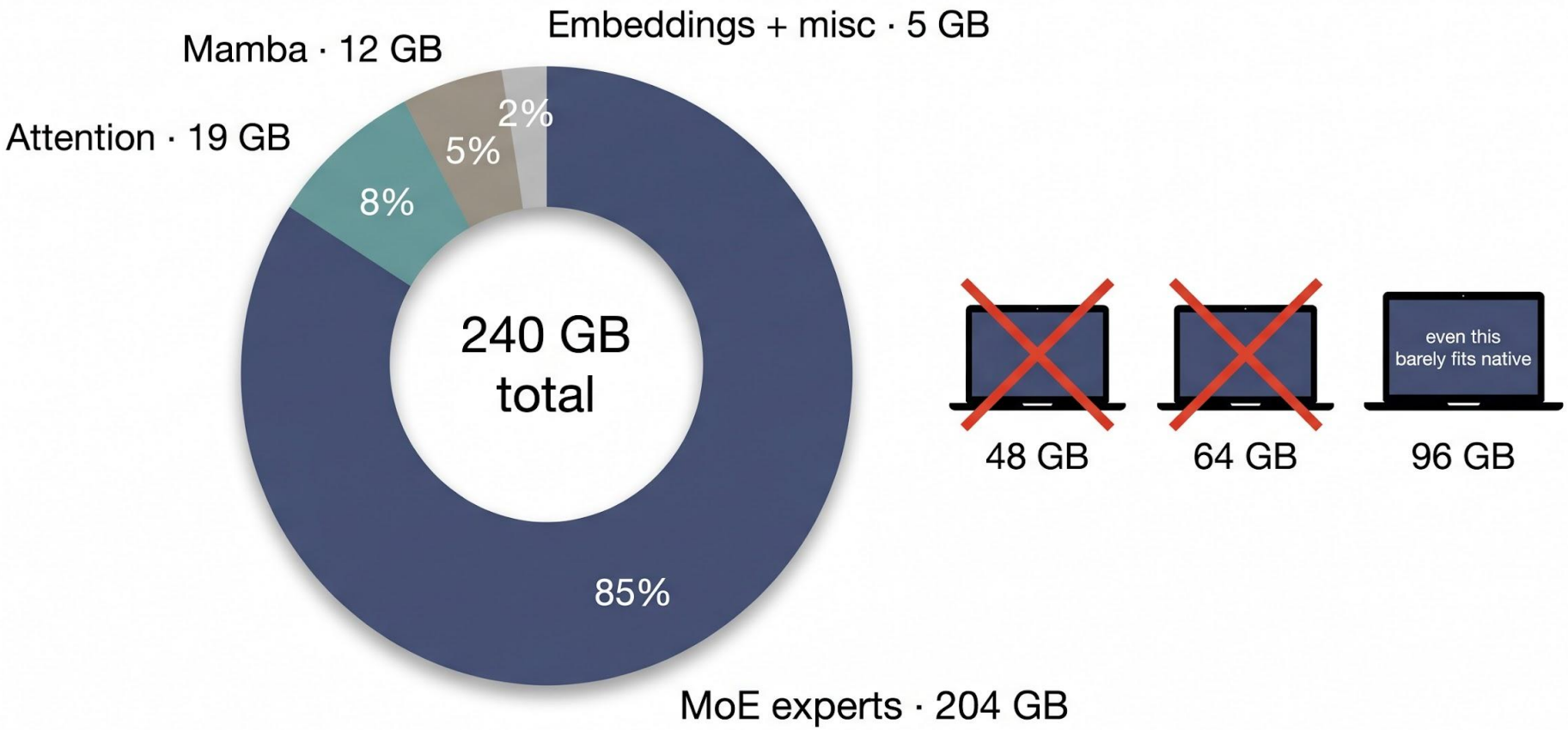
Then ship the finished workload to real infrastructure (NVIDIA DGX).

Local doesn't replace the cluster. Local unblocks the creative loop.

What is Nemotron-3 Super 120B?

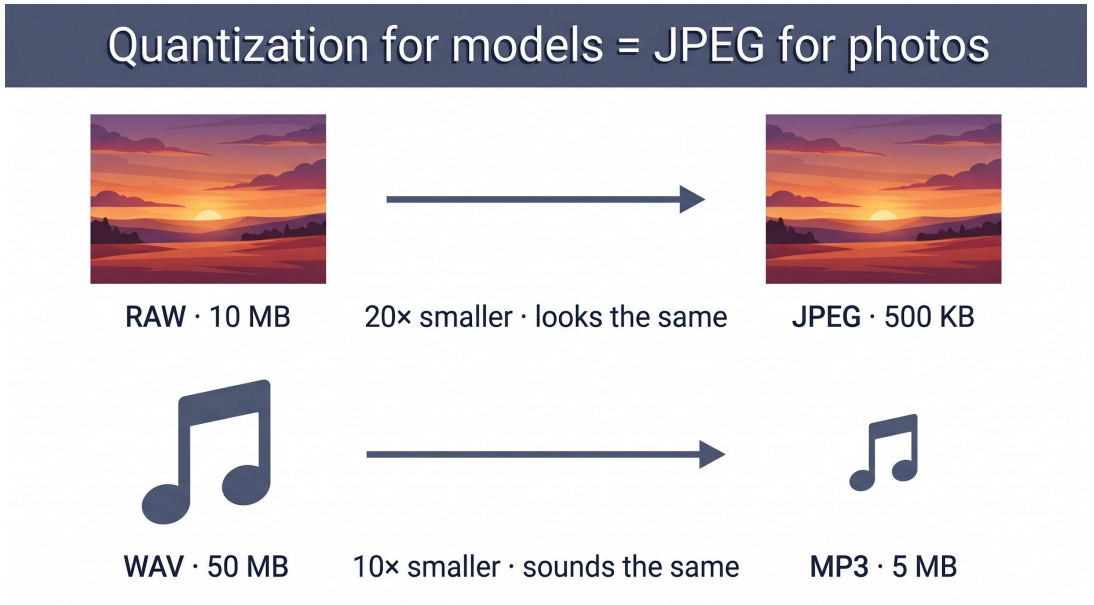
- NVIDIA's new hybrid Mamba-Transformer Mixture-of-Experts model
- 120 B total parameters · ~12 B active per token
- 88 layers · 512 experts per MoE layer · long-context friendly
- "Hybrid" means three layer types working together:
  - Mamba layers — a state-space design, much cheaper than attention for long context
  - Sparse attention — used only where it helps
  - Mixture-of-Experts layers — 512 specialists, a router picks a few per word
- Native size on disk: ~240 GB (BF16)

Where Do 240 GB Go?



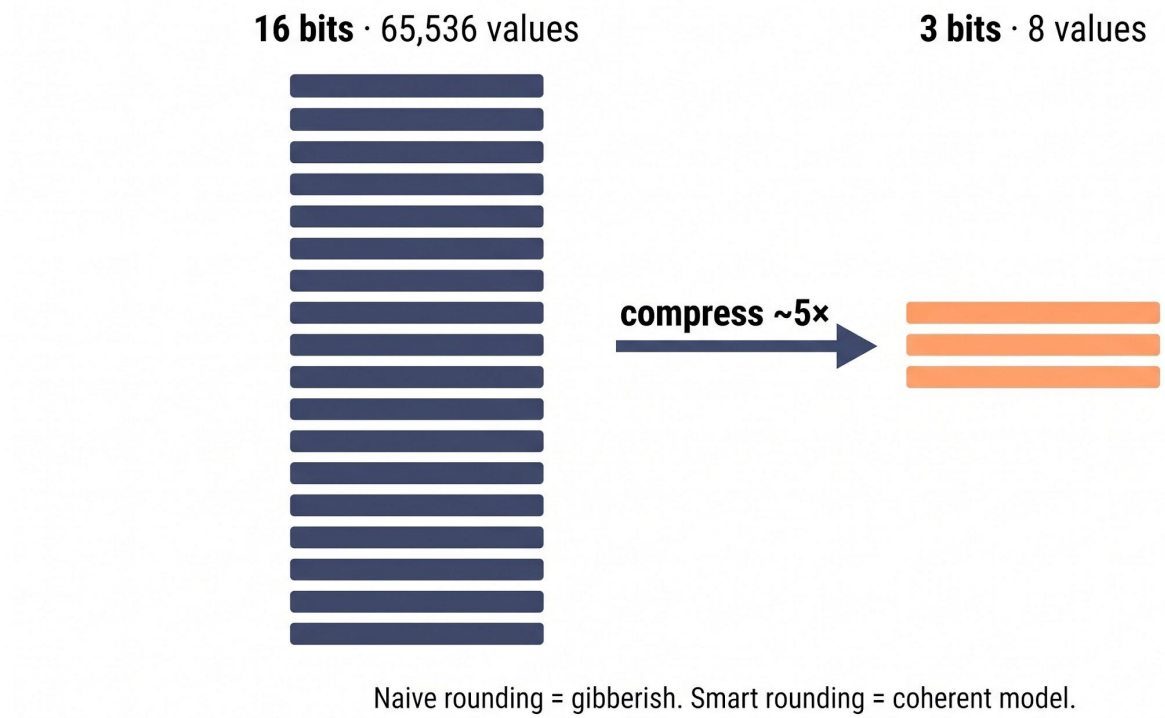
Quantization — The Big Idea (Simple Version)

- Every model weight is just a number. Normally stored with 16 bits each.
- Quantization = use fewer bits per number, accepting tiny rounding errors.
- Our target: 3 bits per weight → about 5× smaller than 16-bit.



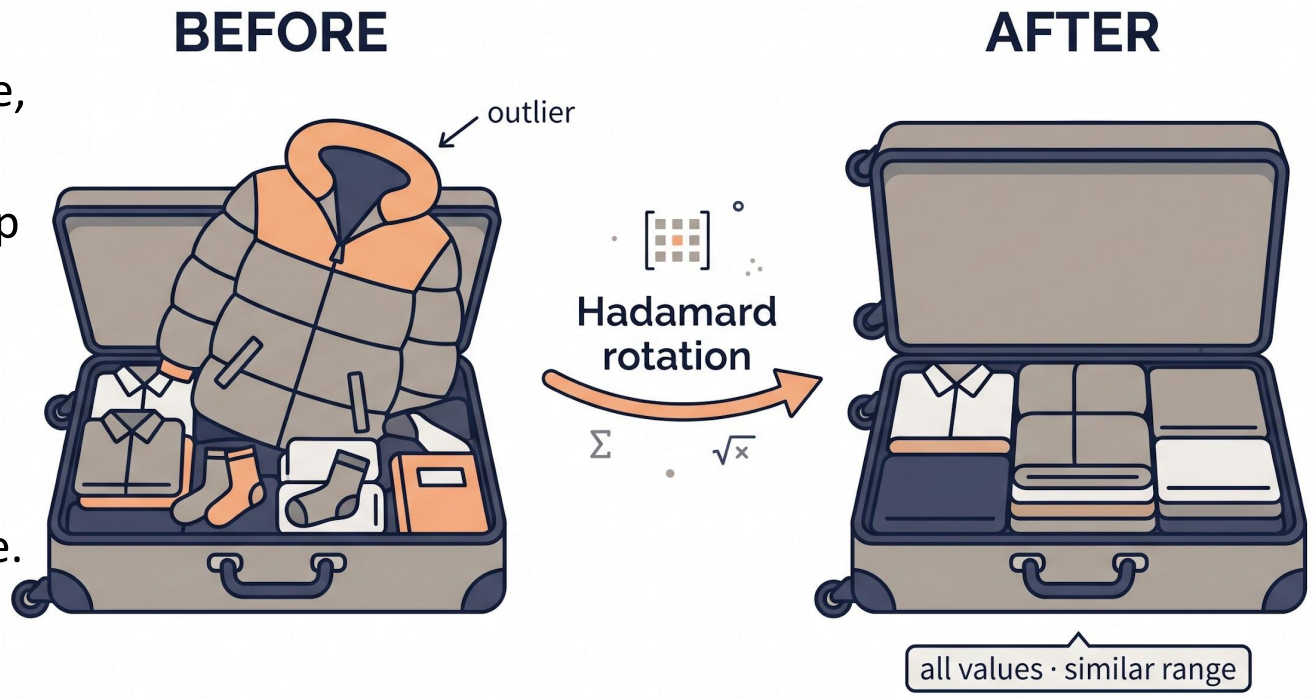
From 16 Bits to 3 Bits

- 16 bits → 65,536 possible values per number
- 3 bits → only 8 possible values per number
- Naive rounding to 8 values destroys the model — it outputs gibberish.
- Smart rounding keeps the model coherent. That's what TurboQuant does.



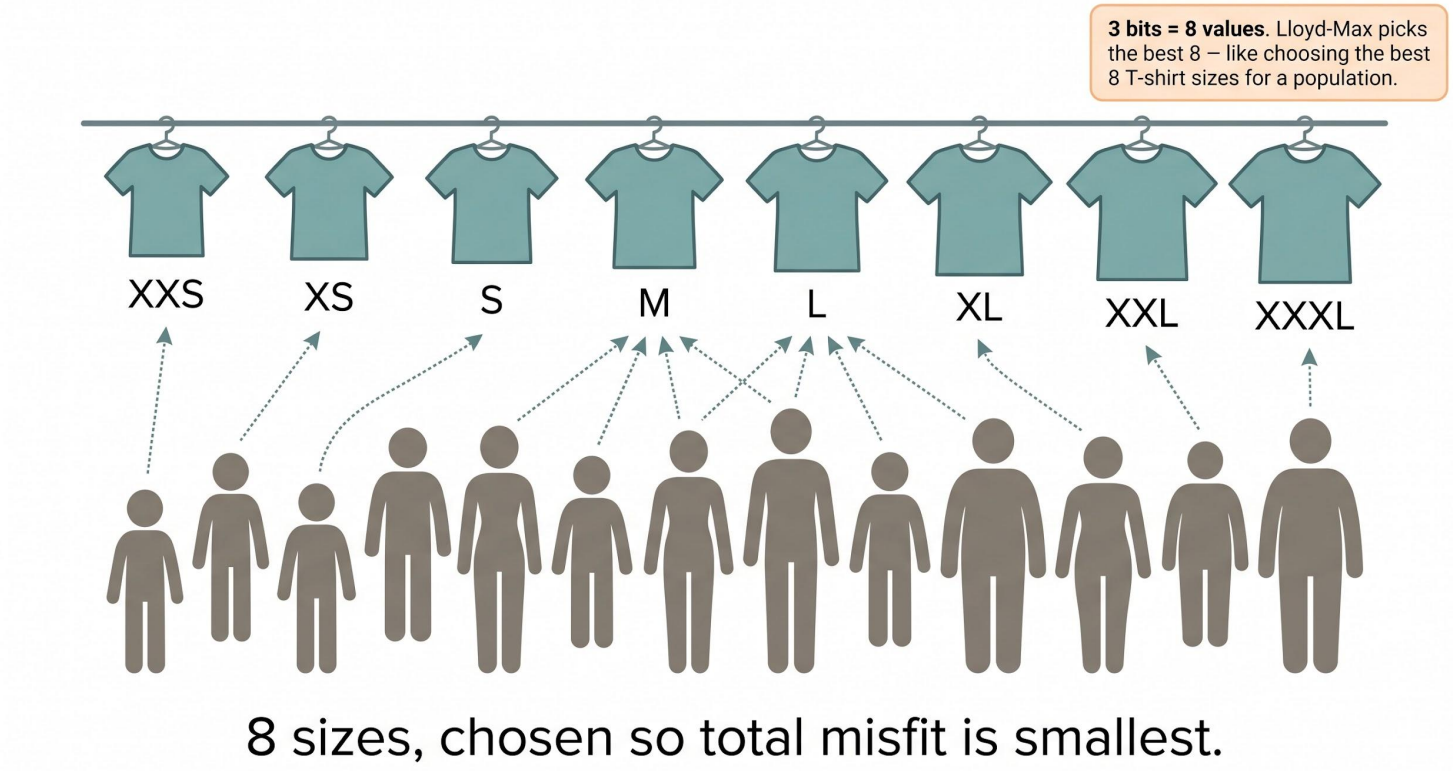
Technique 1: Hadamard Rotation

- Problem: a few weights are much larger than the rest — "outliers".
- Those outliers force us to use a wider range, which wastes precision on everything else.
- Hadamard rotation is the **"pre-folding"** step for model weights:
  - A clever, reversible mathematical reshuffle.
  - Extreme values blend into the pack — every weight now sits in a similar range.
  - Needs zero data to apply. Pure math.



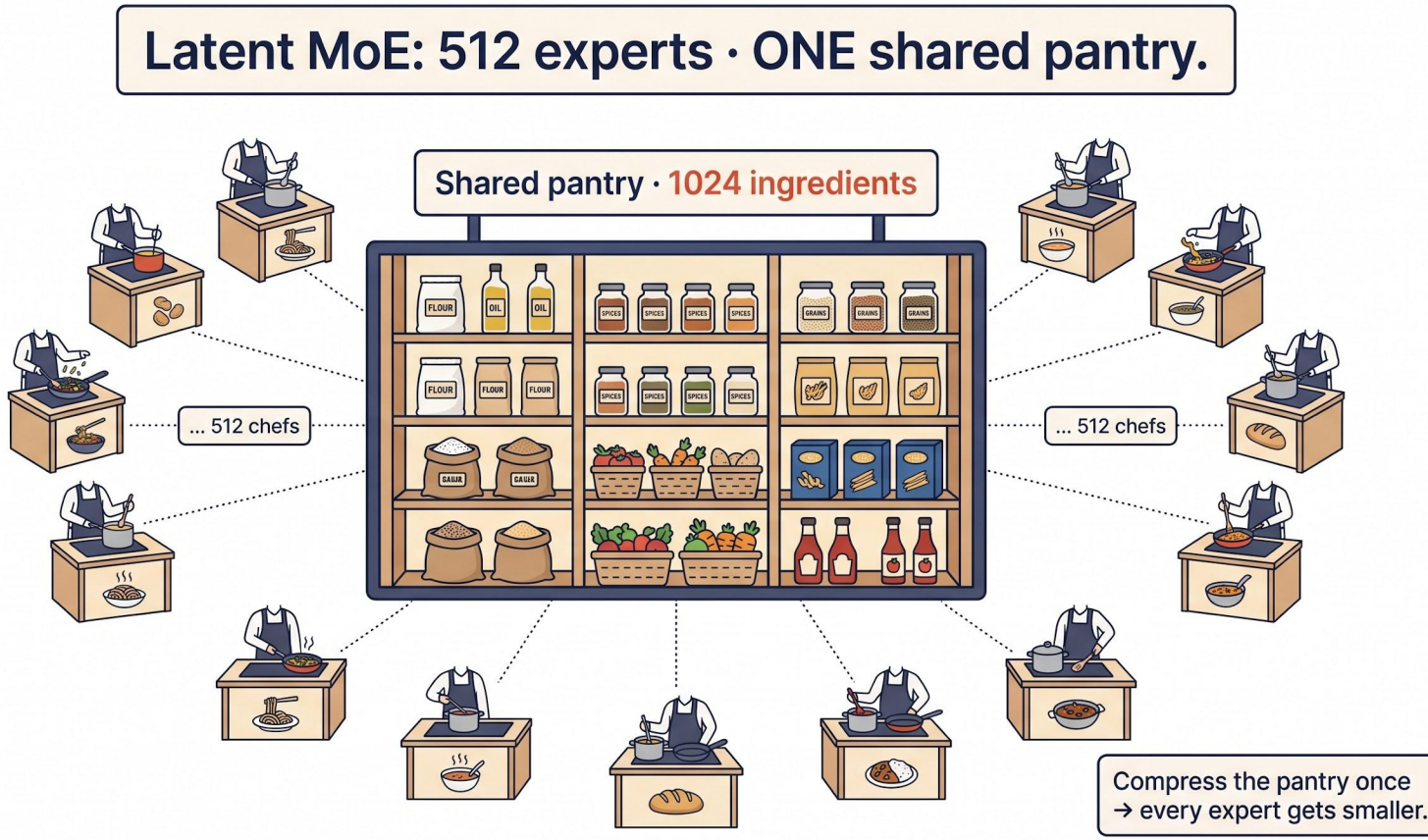
Technique 2: Lloyd-Max Codebooks

- With 3 bits we can only store 8 distinct values. Which 8?
- Lloyd-Max is the math recipe that picks the optimal 8 values:
  - Every weight becomes an index 0-7 — just 3 bits.
  - That index points to one of 8 shared values (the codebook).
- Rotation + Lloyd-Max together = aggressive rounding that preserves quality.



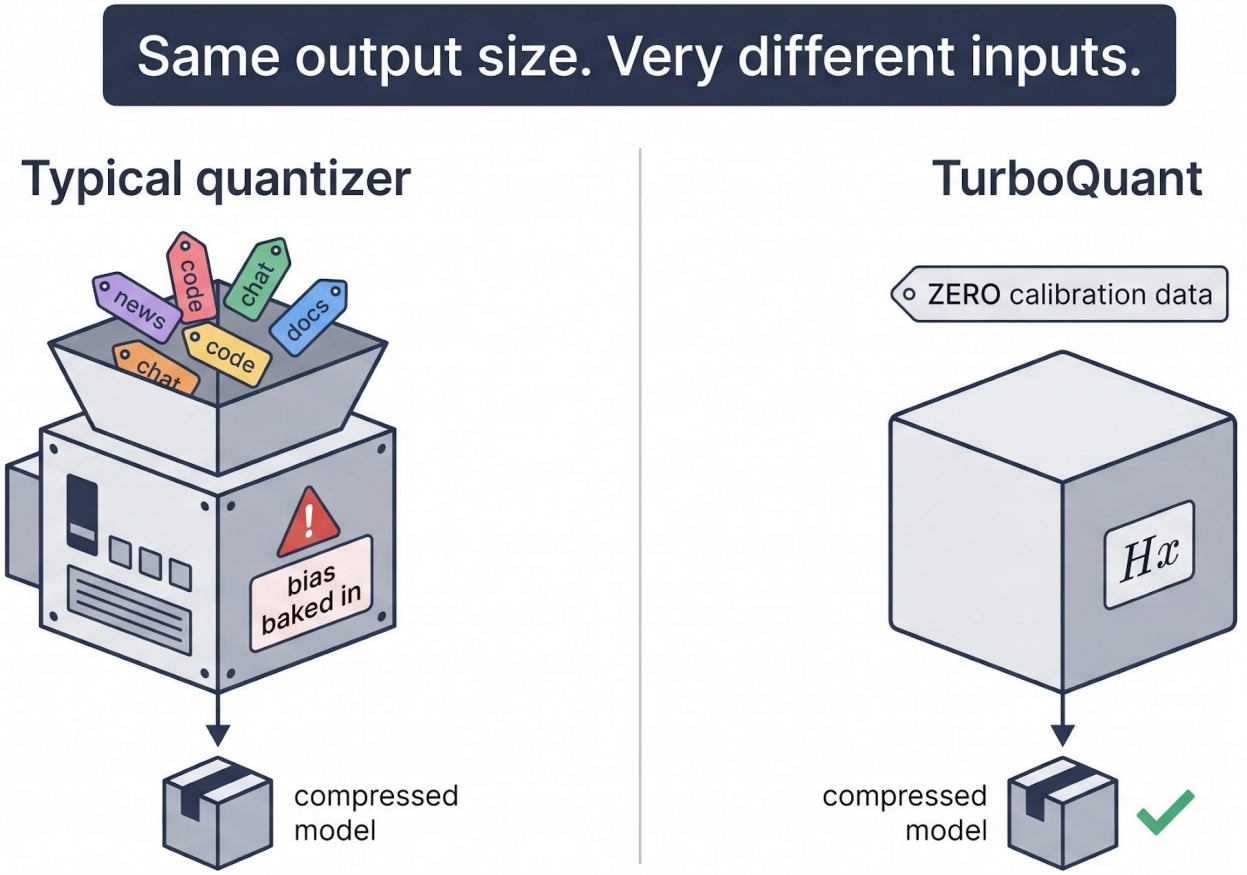
Technique 3: Latent-MoE Quantization

- Reminder — MoE = 512 specialists, a router picks a few per word.
- Nemotron-3 Super uses latent MoE:
  - All 512 experts share a small "pantry" of ingredients (a 1024-dim shared space).
  - Each expert is a light recipe built on top of that shared pantry.
- Quantizing the shared pantry to 3 bits compresses every expert at once.
- This is where most of the **190 GB savings** come from.
- Without latent-MoE compression, the model simply does not fit.



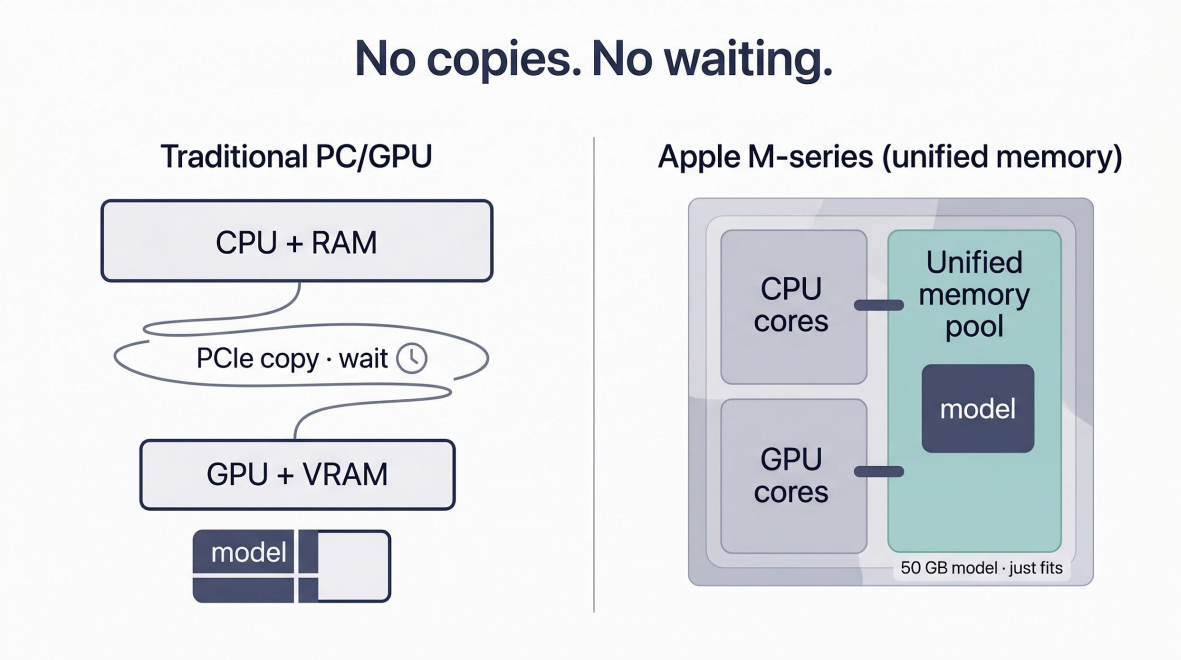
Calibration-Free

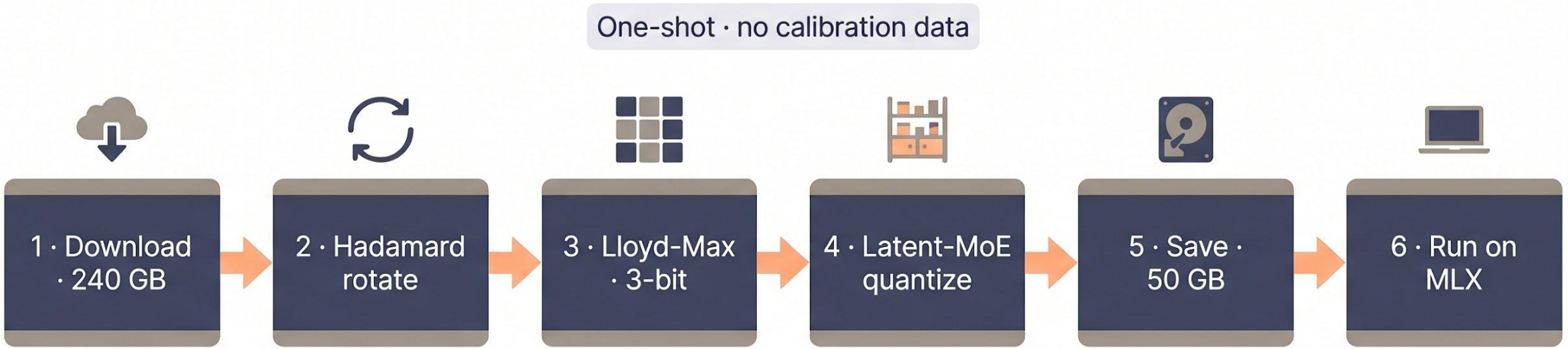
- Most modern quantizers need a calibration dataset:
  - Run sample text through the model to measure "which weights matter".
  - The chosen data silently bakes bias into the quantized model.
  - Pick the wrong data → the model underperforms on corner cases.
- TurboQuant needs zero data — the Hadamard rotation math is universal.
- Result: reproducible, auditable, fast (minutes not hours), domain-neutral.
- Credit: TurboQuant is research from Google (2025). We ported it to MLX.



Why Apple Silicon?

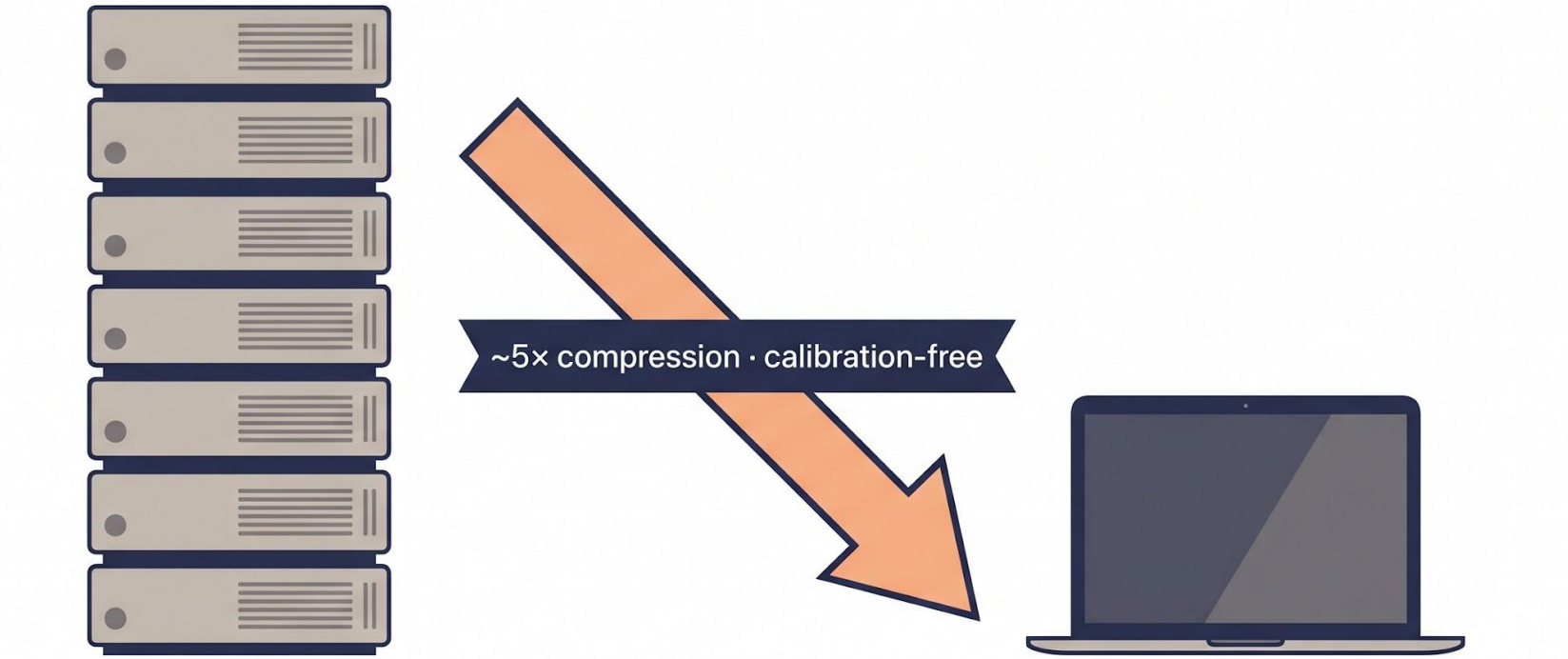
- Unified memory: CPU and GPU share the same RAM pool.
  - No copy-and-wait between "system" and "video" memory.
  - A 50 GB model can actually be loaded on a 64 GB Mac.
- MLX — Apple's ML framework — targets this architecture directly.
- Custom Metal GPU kernels do the 3-bit × 16-bit matmul natively.
- Net result: a 120 B model runs interactively, not just "it loaded".
- Complements NVIDIA infra — use the Mac for dev loop, NVIDIA for production.





TurboQuant-MLX — Putting It Together

Results — Before & After



240 → 50 GB | ~19 tok/s | 54 GB peak RAM | 0 calibration samples

Live Demo — Compression

Live Demo — "Why is the sky blue?"

Resources

- GitHub : github.com/manjunathshiva/turboquant-mlx
- PyPI : pip install turboquant-mlx-full
- Medium : 3-part article series on TurboQuant-MLX
  - https://medium.com/@manjunath.shiva
- MLX : ml-explore.github.io/mlx (Apple)
- Nemotron-3 Super: huggingface.co/nvidia/NVIDIA-Nemotron-3-Super-120B-A12B-BF16

Thank You

LinkedIn